

Ask, Don't Type

How We Built and Shipped a Quiz App by Describing It, Not Coding It

Introduction

Somewhere around final exam season, every student has the same idea: *I wish there was a quiz app for this*. Usually the idea dies right there, because "quiz app" quietly means React components, a database, a hosting bill, and a weekend you don't have. This chapter is about the version of that idea that didn't die — a 200-question practice quiz for CSE146 (Introduction to Cybersecurity) course at North South University for the Summer 2026 semester, covering the OSI model, firewalls, IDS/IPS, penetration testing, and everything else in the Module 4 syllabus, built entirely through a back-and-forth conversation with Claude and shipped to a live Vercel URL the same session.

No line of this app was typed by hand. Every component, every one of the two hundred questions, every CSS token, and the deployment itself came out of natural-language instructions. That is not a magic trick — it's a skill, and like any skill it has a bad-beginner version and a better version. This chapter walks through both, using this exact project as the case study, so you can see the actual prompts, the actual generated code, and the actual moment things almost went wrong.

The Generic Prompt

Here's the prompt almost everyone starts with, because it's the honest first thought:

"Build me a study app for my cybersecurity class."

Nothing about that sentence is wrong to be exact. It's just empty. Read it the way an AI has to read it — as a list of decisions it now must make *for* you, silently, because you didn't make them yourself:

- What framework? React, plain HTML, a Python Flask app, a mobile app? No stack was named, so any of them is a "correct" answer.
- What content? The AI knows nothing about your specific Module 4 material, so it will reach for generic cybersecurity trivia from its own training — CIA triad, common port numbers, maybe something about phishing. None of it will match what your professor actually taught, which defeats the entire purpose of exam prep.
- How many questions, and in what format? Five? Fifty? Multiple choice, flashcards, an essay grader? Nothing constrains it.
- Where does it run, and how do you know it's "done"? There's no way to check the output against anything, because nothing was specified to check it against.

Run that prompt and you'll get *something* — a single HTML file with eight generic questions about firewalls, maybe, styled in whatever default aesthetic the model reaches for that day. It will look like a study app. It will not be one, because the questions won't match your course, the format won't fit how you actually want to review, and there's nowhere to put it so classmates could use it too. This is the exact failure mode the Snake-game chapter describes as getting "mush" — except here the mush isn't just aesthetically generic, it's *factually wrong for your exam*, which is a much more expensive kind of vague.

Improving the Prompt

The fix isn't a longer prompt for its own sake — it's a prompt that removes every decision you don't want the AI making on its own. A few techniques did the actual work here:

Borrow a structure, like CRAFT. CRAFT (Context, Requirements, Aesthetics, Features, Test & Tweak) — scales up fine from to a real app. Context became the full tech stack and hosting target. Requirements became the exact question format and count. Aesthetics became a specific color palette tied to the student's existing study materials. Features became the topic filter, timer, and review mode. Test & Tweak became the build-and-click-through pass described later in this chapter.

Set hard content boundaries. For a quiz that's supposed to mirror one specific course, the single most important instruction isn't about code at all — it's about a fence around the AI's own knowledge: *use only the source material provided; do not introduce outside facts*. If you leave a LLM like Claude to itself without explicitly telling it to keep it limited to your supplied course material, then it will happily fill gaps with plausible-sounding cybersecurity trivia it picked up in training, and "plausible-sounding" is exactly the kind of wrong answer that costs a real exam point. Pasting in the actual lecture content and saying "only this" turns a knowledge generator into a formatting and organizing tool for content you already trust.

Name the stack precisely, including the constraints. "Build a web app" leaves the framework, the hosting model, and the backend all open. "Next.js App Router, TypeScript, Tailwind, fully client-side, zero environment variables, deployable to Vercel with zero config" leaves nothing open. Every one of those words removed a decision the AI would otherwise have made for you — and a couple of those decisions (no backend, no env vars) were really decisions about *cost and complexity*, not aesthetics.

Give it a schema before you give it content. This is the technique that mattered most for a data-heavy app like this one, and it's the one thing the game-building chapter doesn't need much of, because a game's state lives in code, not in two hundred hand-checkable records. Before asking for a single question, the prompt specified the exact TypeScript shape every question object had to satisfy — field names, allowed types, a strict four-option tuple for multiple choice. A schema does two jobs at once: it prevents structural hallucination (the model can't quietly invent a fifth answer option or drop the `explanation` field on question 147), and it gives you, the human, something mechanical to check against afterward instead of reading two hundred objects by eye.

Write an acceptance checklist into the prompt itself. The original prompt ended with a literal checklist — exact question counts, topic coverage, "no outside facts," "next build passes cleanly," "deploys with zero config." That checklist did double duty as instructions to the AI and as a verification script for the human

afterward. It's the same idea as CRAFT's "Test & Tweak," just written down in advance instead of improvised after the fact.

The Master Prompt

Here is the actual prompt — trimmed only in one place, noted below — that produced the app in this repository.

```
Build a self-contained web quiz app for practicing CSE146 (Introduction to Cybersecurity) Module 4 + Module 4 Supplement content, deployed to Vercel. This is exam-prep software for a single student to drill 200 practice questions before a quiz. No backend, no database, no authentication - everything runs client-side and is deployable as a static/serverless Next.js app on Vercel's free tier with zero configuration and zero environment variables.
```

```
Total question bank: 200 questions
```

- 150 Multiple Choice Questions (MCQ) - 4 options each, exactly 1 correct answer.
- 50 Fill-in-the-Blank questions - a sentence with a key term/phrase removed, graded by matching the typed answer (case-insensitive, trimmed, allow minor whitespace variance; accept a short list of acceptable synonyms/aliases per question).

```
You are responsible for authoring the actual 200 questions. Use ONLY the source material provided below. Do not introduce outside cybersecurity facts, vendor products, or terminology that isn't present in that section. Several definitions are quoted verbatim in the source material - reuse that exact phrasing, since the real quiz tests recognition of those exact phrases.
```

TECH STACK

```
Next.js (App Router) + TypeScript, Tailwind CSS, no database, no API keys, no environment variables. All question data lives in a local data/questions.ts file bundled with the app. State managed with React state + localStorage, so a page refresh mid-quiz doesn't lose progress.
```

```
[... the prompt continued with roughly 2,500 words of the actual Module 4 lecture content - the OSI model table, TCP vs. UDP guidance, all eight firewall types, stateful inspection mechanics, IDS/IPS definitions, DLP/SIEM, penetration testing steps, risk management, CSIRT, and Cisco ISE/TrustSec - pasted in verbatim as the only permitted source of facts. It's omitted here for length, but its presence is the whole point: every one of the 200 questions below had to trace back to a sentence in that block. ...]
```

DATA SCHEMA

```
export type Question =  
  | {  
    id: string;  
    type: "mcq";
```

```

    topic: string;
    prompt: string;
    options: [string, string, string, string];
    correctIndex: 0 | 1 | 2 | 3;
    explanation: string;
  }
  | {
    id: string;
    type: "fill-blank";
    topic: string;
    prompt: string;
    acceptableAnswers: string[];
    explanation: string;
  };

```

Generate exactly 150 objects with type "mcq" and 50 with type "fill-blank", spread across 18 topic tags.

APP FEATURES

Home screen with a topic filter (18 topics, default all selected), a quiz-length selector (10 / 25 / 50 / All), and a timed-mode toggle. Quiz screen with a progress bar, running score, immediate per-question review with the correct answer and a short explanation, and a countdown that auto-submits as wrong on timeout. Results screen with the final score, a breakdown by topic, a list of missed questions, and "Retry All" / "Retry Missed Only" buttons. Persist quiz-in-progress state to localStorage.

DEPLOYMENT

next build must succeed with no errors before this is done. No environment variables, no serverless functions. Include a README with local dev and both Vercel deploy paths (CLI and GitHub import). After building, run the production build locally and click through a full quiz end-to-end before calling this complete.

ACCEPTANCE CHECKLIST

- Exactly 150 MCQ + 50 fill-in-the-blank, all grounded in the source material, no outside facts
- Questions proportionally cover all 18 topics
- A meaningful subset reuse exact quoted definitions
- No duplicate or near-duplicate questions
- Topic filtering, timed mode, score tracking, and review mode all work
- Progress persists across a refresh
- Fully responsive on mobile
- next build passes cleanly
- Deploys to Vercel with zero config

Notice what this prompt refuses to leave open, compared to "build me a study app": the exact stack, the exact question counts and grading rules, the exact data shape, the exact feature list, and an exact, checkable definition of done. Nothing in there is exotic — it's just decisions, made once, in writing, instead of made silently by whichever model happens to answer.

The Generated Output

The schema in the prompt is not decoration — it's the actual TypeScript type the generated codebase ships with, unedited, in `data/questions.ts`:

```
export type McqQuestion = {
  id: string;
  type: "mcq";
  topic: string;
  prompt: string;
  options: [string, string, string, string];
  correctIndex: 0 | 1 | 2 | 3;
  explanation: string;
};

export type FillBlankQuestion = {
  id: string;
  type: "fill-blank";
  topic: string;
  prompt: string;
  acceptableAnswers: string[];
  explanation: string;
};

export type Question = McqQuestion | FillBlankQuestion;
```

And here's one of the two hundred generated question objects that has to satisfy it — question one, topic "osi," pulled straight from the file:

```
{
  id: "osi-01",
  type: "mcq",
  topic: "osi",
  prompt:
    "Which OSI layer is responsible for logical addressing and routing, using
    protocols like IP, ICMP, OSPF, and BGP?",
  options: ["Physical", "Data Link", "Network", "Transport"],
  correctIndex: 2,
  explanation:
    "Layer 3 (Network) handles logical addressing and routing via IP, ICMP, OSPF,
    and BGP (§4.1 OSI Model table).",
},
```

Every field lines up with the schema, and the fact itself — Layer 3, logical addressing, IP/ICMP/OSPF/BGP — comes straight out of the pasted lecture table, not from general knowledge. That's what "use only the source material" and "generate exactly this shape" bought together: content you can trust and structure you can check with code instead of by eye. In fact, that's literally what happened — a small script counted the generated

objects and confirmed exactly 150 "mcq" and 50 "fill-blank" entries with 200 unique IDs before this was considered finished.

The grading logic that reads that data is just as plain. Here's the whole rule for scoring a fill-in-the-blank answer, from `lib/quiz.ts`:

```
export function normalizeAnswer(raw: string): string {
  return raw.trim().toLowerCase().replace(/s+/g, " ").replace(/[,!?:;]+$/g, "");
}

export function isFillBlankCorrect(question: Question, raw: string): boolean {
  if (question.type !== "fill-blank") return false;
  const normalized = normalizeAnswer(raw);
  if (!normalized) return false;
  return question.acceptableAnswers.some(
    (a) => normalizeAnswer(a) === normalized
  );
}
```

Nothing clever — trim it, lowercase it, strip trailing punctuation, and check it against a short list of acceptable phrasings. It works precisely because the prompt described that exact behavior ("case-insensitive, trimmed, allow minor whitespace variance") instead of just saying "grade the answers" and hoping for something reasonable.

The Deployment Workflow

Writing the app is half the job; a Next.js project sitting on a laptop is still just a folder. Getting it onto the internet followed a short, unglamorous pipeline:

1. **Verify locally, twice.** `npm run build` first, to catch type errors and confirm a clean production build with no warnings. Then `npm run build && npm run start` to run the actual production server and click through a full quiz — every question type, the timer, the topic filter, a refresh mid-quiz to confirm progress really does survive in `localStorage`, and the results screen's retry flows. This step caught a real bug: the "None" button on the topic filter left one topic silently checked instead of clearing the selection, which only showed up by actually clicking it, not by reading the code.
2. **Turn the folder into a repository.** `git init`, stage everything except `node_modules` and `.next` (already excluded by the generated `.gitignore`), and `commit`. This step matters even for a solo project — it's the difference between "a folder" and "a thing with history you can roll back."
3. **Deploy.** For this project, that meant authenticating the Vercel CLI once (`vercel whoami` confirmed an existing login) and running `vercel -prod`. Vercel auto-detected the Next.js project, ran `next build` on its own servers, and handed back a live production URL — no config file, no environment variables, no manual server setup. The equally valid alternative, better suited to a project that will keep changing, is

pushing the repository to GitHub and importing it on vercel.com once: after that, every future git push triggers a new deployment automatically. Both paths end at the same place — a public URL — they just differ in whether a human or a webhook clicks "deploy" going forward.

4. **Check the live site, not just the local one.** The deployed URL was reopened in a browser afterward and clicked through again. A build that succeeds is not the same claim as an app that works; only one of those two things was actually verified against the live deployment.

None of these steps required understanding how Vercel's build infrastructure works internally. They required knowing what "done" looks like clearly enough to check for it — which is the same skill the master prompt's acceptance checklist was for, just applied one more time after the code existed instead of before.

Remarks

Nothing in this chapter is about AI being clever. The model didn't decide what a CSE146 exam covers, neither it decide that fill-in-the-blank answers should tolerate trailing punctuation, or decide that "None" should actually mean none. A human made every one of those calls, wrote them down precisely enough to hand off, and then — this is the part that's easy to skip — actually opened the running app and checked. The AI is the engine: fast, tireless, and willing to write two hundred structurally identical questions without complaint. The human is still the architect who decides what "correct" means for this course, and the reviewer who catches it when the engine's output looks right but isn't. Skip the second half of that sentence, and you don't get a study tool — you get two hundred confident-sounding guesses with a nice color scheme. Keep it, and the gap between "I wish there was an app for this" and a live URL you can hand to your whole class shrinks down to the length of one honest conversation.

**Claude Opus 5 and Claude Sonnet 5 AI Models were used to write this chapter*